ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

TOÁN ỨNG DỤNG VÀ THỐNG KÊ

ĐỒ ÁN 1

Ma Trận và Cơ Sở của Tính Toán Khoa Học

Giảng viên hướng dẫn: ThS. Võ Nam Thục Đoan
ThS. Lê Nhựt Nam

Nhóm 02:

Phạm Minh Trọng	24120151
Đào Tiến Đạt	24120033
Bùi Nhật Bảo	24120167
Trịnh Kim Mai	24120199
Trần Công Quang	24120221

Thành phố Hồ Chí Minh, 4/2026

Mục lục

Lời Nói Đầu **3**

Lời Cam Đoan **4**

Bảng Phân Công Công Việc **5**

1 Phần 1: Phép khử Gauss và Các Ứng Dụng **6**

1.1 Cơ sở lý thuyết 6

1.1.1 Hệ phương trình tuyến tính và Ma trận tăng cường 6

1.1.2 Các phép biến đổi dòng sơ cấp 6

1.1.3 Dạng bậc thang (REF) và Bậc thang rút gọn (RREF) 6

1.1.4 Khử Gauss có chọn phần tử chốt (Partial Pivoting) 7

1.1.5 Tính định thức qua khử Gauss 7

1.1.6 Ma trận nghịch đảo bằng Gauss-Jordan 7

1.1.7 Hạng ma trận và cơ sở 7

1.2 Thuật toán khử Gauss (Partial Pivoting) 7

1.3 Triển khai và cài đặt chi tiết (Phân tích mã nguồn) 8

1.3.1 Hàm `gaussian_elimination(A, b)` 8

1.3.2 Hàm `back_substitution(U, c)` 9

1.3.3 Hàm `inverse(A)` 9

1.3.4 Hàm `determinant(A)` 10

1.3.5 Hàm `rank_and_basis(A)` 10

1.3.6 Hàm `verify_solution(A, x, b)` 11

1.4 Đánh giá tiêu chí hoàn thành 11

2 Phần 2: Phân Rã Ma Trận và Trực Quan Hóa với Manim **13**

2.1 Chéo hóa ma trận (Matrix Diagonalization) 13

2.1.1 Cơ sở lý thuyết 13

2.1.2 Thuật toán cài đặt 13

2.2 Phân rã Giá trị suy biến (SVD) 14

2.2.1 Cơ sở lý thuyết 14

2.2.2 Phân rã tĩnh bằng phép Xoay Jacobi (Jacobi Eigenvalues) 14

2.3 Ứng dụng và Trực quan hóa với Manim 15

2.3.1 Ứng dụng 15
2.3.2 Trực quan hóa 15
2.4 Đánh giá tiêu chí hoàn thành 17

3 Phần 3: Giải Hệ Phương Trình và Phân Tích Hiệu Năng 18
3.1 Các phương pháp cấu hình và giải hệ 18
3.2 Đo lường thời gian thực thi (Performance Benchmark) 18
3.3 Đánh giá tính ổn định số học (Condition Number) 19

4 Kết Luận 20
4.1 Tóm tắt kết quả đạt được 20
4.2 Khó khăn gặp phải 20
4.3 Bài học rút ra 21

Lời Nói Đầu

Trong bối cảnh bùng nổ của trí tuệ bhân tạo và khoa học dữ liệu, nền tảng đại số tuyến tính và tính toán Khoa học đóng vai trò đặc biệt quan trọng, là điều kiện trong thể thiếu trong mọi hệ thống học máy, nén dữ liệu và tối ưu hóa hiện đại. Đồ án **"Ma Trận và Cơ Sở của Tính Toán Khoa Học"** không chỉ đơn thuần là một bài tập lớn, mà còn là cơ hội để chúng em tiếp cận được những kiến thức mới hơn, biết được ứng dụng của đại số tuyến tính vào trong cuộc sống, xây dựng được nền tảng để bước vào chuyên ngành

Dù đã nỗ lực hết sức và nghiêm túc trong công việc hoàn thiện đồ án , song do những giới hạn về mặt thời gian cũng như kinh nghiệm thực tiễn, báo cáo này chắc chắn vẫn còn những thiếu sót nhất định. Chúng em rất mong nhận được sự đánh giá, góp ý quý báu từ quý Thầy Cô để nhóm có thể đúc kết thêm kinh nghiệm và hoàn thiện bản thân hơn trong những chặng đường học tập tiếp theo.

Chúng em xin chân thành cảm ơn!

Lời Cam Đoan

Chúng em, Nhóm 02, xin cam đoan đây là đồ án tự nghiên cứu và cài đặt của cả nhóm. Các dòng mã nguồn, phân tích thuật toán, cấu trúc dữ liệu và logic xử lý đều được nhóm tự cài đặt từ đầu bằng Python theo đúng yêu cầu của đồ án, hoàn toàn không sử dụng các hàm có sẵn của các thư viện như NumPy/SciPy để gian lận

Mọi kết quả kiểm thử, số liệu thực nghiệm, video trực quan hóa bằng Manim được trình bày trong báo cáo này là hoàn toàn trung thực, phản ánh đúng kết quả thực thi từ mã nguồn của nhóm. Nếu có bất kỳ sự vi phạm đạo đức học thuật nào, chúng em xin hoàn toàn chịu trách nhiệm trước giảng viên hướng dẫn và Khoa Công nghệ Thông tin, Trường Đại học Khoa học Tự nhiên, ĐHQG-HCM.

Đại diện nhóm

Phạm Minh Trọng

Bảng Phân Công Công Việc

STT	Họ và Tên	Nhiệm vụ phân công	Hoàn thành
1	Phạm Minh Trọng **(24120151 - Leader)**	Quản lý tiến độ, dựng kịch bản, trực quan hóa bằng Manim (part2), hoàn thiện báo cáo	100%
2	Đào Tiến Đạt **(24120033)**		100%
3	Bùi Nhật Bảo **(24120167)**	Dựng kịch bản, trực quan hóa bằng Manim (part2), hoàn thiện báo cáo	100%
4	Trịnh Kim Mai **(24120199)**		100%
5	Trần Công Quang **(24120221)**	Cài đặt chéo hóa và SVD bằng Python,	100%

Bảng 1: Chi tiết phân công nhiệm vụ của các thành viên trong nhóm

1 Phần 1: Phép khử Gauss và Các Ứng Dụng

1.1 Cơ sở lý thuyết

1.1.1 Hệ phương trình tuyến tính và Ma trận tăng cường

Xét hệ phương trình tuyến tính có dạng tổng quát:

$$Ax = b \tag{1}$$

Trong đó:

- $A \in \mathbb{R}^{m \times n}$: Ma trận hệ số.
- $x \in \mathbb{R}^n$: Vector ẩn số.
- $b \in \mathbb{R}^m$: Vector vế phải.

Để giải hệ phương trình, ta thiết lập Ma trận tăng cường $[A|b]$ bằng cách ghép vector b vào bên phải ma trận A.

1.1.2 Các phép biến đổi dòng sơ cấp

Thuật toán dựa trên ba phép biến đổi cơ bản không làm thay đổi tập nghiệm của hệ phương trình:

1. Hoán đổi dòng: $R_i \leftrightarrow R_j$
2. Nhân dòng với một hằng số khác 0: $R_i \leftarrow c \cdot R_i \quad (c \neq 0)$
3. Cộng vào một dòng bội số của dòng khác: $R_i \leftarrow R_i + c \cdot R_j$

1.1.3 Dạng bậc thang (REF) và Bậc thang rút gọn (RREF)

- **Dạng bậc thang (Row Echelon Form - REF):** Các dòng bằng 0 nằm dưới cùng; phần tử khác 0 đầu tiên của một dòng (phần tử chốt - pivot) luôn nằm bên phải phần tử chốt của dòng phía trên nó.
- **Dạng bậc thang rút gọn (Reduced Row Echelon Form - RREF):** Là dạng REF nhưng mọi phần tử chốt đều bằng 1 và là phần tử khác 0 duy nhất trong cột chứa nó.

1.1.4 Khử Gauss có chọn phần tử chốt (Partial Pivoting)

Để tăng độ ổn định số học và tránh lỗi chia cho 0, ta chọn phần tử chốt có giá trị tuyệt đối lớn nhất trong cột hiện tại trước khi thực hiện phép khử:

$$|a_{pk}| = \max_{i \geq k} |a_{ik}| \tag{2}$$

1.1.5 Tính định thức qua khử Gauss

Sau khi đưa ma trận A về dạng tam giác trên U bằng phép khử Gauss với s lần hoán đổi dòng, định thức được tính theo công thức:

$$\det(A) = (-1)^s \cdot \prod_{i=1}^{n} u_{ii} \tag{3}$$

1.1.6 Ma trận nghịch đảo bằng Gauss-Jordan

Định lý: Ma trận $A \in \mathbb{R}^{n \times n}$ khả nghịch khi và chỉ khi $\det(A) \neq 0$.

Ma trận nghịch đảo A^{-1} được tìm bằng cách thực hiện các phép biến đổi dòng sơ cấp đồng thời trên ma trận ghép $[A|I_n]$ cho đến khi ma trận bên trái đạt dạng đơn vị:

$$[A|I_n] \xrightarrow{\text{biến đổi dòng}} [I_n|A^{-1}] \tag{4}$$

1.1.7 Hạng ma trận và cơ sở

Định nghĩa: Hạng của ma trận A, ký hiệu là $\text{rank}(A)$, bằng số dòng khác 0 trong dạng REF của A, hoặc tương đương, bằng số cột pivot. Từ dạng RREF, ta xác định được ba không gian quan trọng:

- **Không gian cột $\mathcal{C}(A)$:** Sinh bởi các cột pivot của A gốc.
- **Không gian dòng $\mathcal{R}(A)$:** Sinh bởi các dòng khác 0 trong RREF.
- **Không gian nghiệm $\mathcal{N}(A)$:** Tập nghiệm của hệ thuần nhất $Ax = 0$.

1.2 Thuật toán khử Gauss (Partial Pivoting)

Quy trình thực hiện được nhóm cài đặt gồm hai giai đoạn chính:

- **Giai đoạn 1: Khử xuôi (Forward Elimination)**

1. Khởi tạo ma trận tăng cường $M = [A|b]$.
2. Với mỗi cột k:
3. Tìm dòng p chứa phần tử có trị tuyệt đối lớn nhất kể từ dòng k trở xuống.
4. Hoán đổi dòng k và dòng p.
5. Với mỗi dòng $i > k$: Thực hiện phép biến đổi dòng để triệt tiêu các phần tử bên dưới phần tử chốt M_{kk}.

- **Giai đoạn 2: Thế ngược (Back Substitution)**
  Tìm nghiệm x_i theo công thức truy hồi từ dòng cuối cùng lên:

$$x_i = \frac{M_{i,n+1} - \sum_{j=i+1}^{n} M_{ij} \cdot x_j}{M_{ii}} \tag{5}$$

1.3 Triển khai và cài đặt chi tiết (Phân tích mã nguồn)

Trong phần này, nhóm tiến hành xây dựng các thuật toán từ đầu sử dụng Python cơ bản. Để đảm bảo triệt tiêu sai số làm tròn (round-off error), mọi phép tính phân số được bảo toàn nhờ đối tượng `fractions.Fraction`. Gồm các hàm cụ thể như sau:

1.3.1 Hàm `gaussian_elimination(A, b)`

Hàm đóng vai trò nền tảng, tiến hành biến đổi ma trận A và b về dạng tương đương bậc thang (REF).

- **Cơ chế hoạt động:** Ở mỗi cột, thuật toán tìm phần tử có trị tuyệt đối lớn nhất từ dưới lên (Partial Pivoting) rồi đưa lên vị trí chốt thông qua hoán vị dòng, bảo vệ hệ thống khỏi lỗi chia số 0.
- **Cơ chế tích hợp giải nghiệm sâu:** Thuật toán tối ưu hóa mặt luồng chéo dòng bằng việc tự động trích xuất U, c_result để đẩy qua hàm `back_substitution` nội bộ, giúp xuất khẩu ra ngoài một bộ ‘M, x, s‘ khép kín một cách hoàn hảo.

```
# Ky thuat Partial Pivoting: Tim phan tu chot lon nhat tren cot hien tai
p = r
for i in range(r + 1, n):
    if abs(M[i][c_idx]) > abs(M[p][c_idx]):
        p = i

```

```
if p != r:
    M[r], M[p] = M[p], M[r]  # Hoan vi hang de giu su on dinh
    s += 1                   # Ghi nhan so lan hoan vi cho dinh thuc
```

1.3.2 Hàm `back_substitution(U, c)`

Tiến hành truy hồi nghiệm từ chốt cuối cùng lên trên sau khi khử về dạng bậc thang.

- **Tự động quét hạng động (rank):** Thuật toán tự động quét tìm dòng chứa giá trị $\neq 0$ để định lượng chính xác rank, cho phép ứng phó với bất kỳ định dạng bậc thang nào bị khuyết phần tử.
- **Cấu trúc tham số linh hoạt:** Với tình huống vô số nghiệm, nhóm sử dụng `Dictionary` để biểu diễn các hệ số tham số tổng quát dưới dạng ánh xạ.

```
# Bieu dien cac he so tham so tong quat duoi dang anh xa Dictionary
# VD: x[j] = {'const': gia_tri, index_bien_tu_do: he_so, ...}
x_expr = [{} for _ in range(n_cols)]

for j in free_cols:
    x_expr[j] = {'const': Fraction(0), j: Fraction(1)} # Kich hoat bien tu do
```

1.3.3 Hàm `inverse(A)`

Tìm nghịch đảo ma trận bằng cách biến đổi đồng thời trên vùng không gian mở rộng.

- **Cơ chế hoạt động:** Bắt đầu từ trục tọa độ $[A|I_n]$, thuật toán chuẩn hóa đồng độ cho pivot về 1, sau đó tiến hành quét sạch (trừ khử) toàn bộ các phần tử ở DƯỚI lẫn TRÊN trục pivot, cưỡng ép phần A về ma trận I, đẩy A^{-1} sang phải.
- **Bắt lỗi suy biến:** Ngay tại vòng lặp, nếu bắt gặp toàn cột bằng 0 hoặc $\det(A) = 0$, hệ thống rẽ nhánh `None` tức thì để ngăn chặn hiện tượng crash hệ thống.

```
# Khu ca cac dong tren lan duoi de dua ma tran truc trai ve don vi I
for i in range(n):
    if i == k: continue
    factor = M[i][k]
    for j in range(2 * n):
        M[i][j] -= factor * M[k][j]
```

```
    M[i][k] = Fraction(0)  # Tru gian tiep luon dat sai so ve 0 tuyet doi
```

1.3.4 Hàm `determinant(A)`

Hàm này áp dụng kết quả của phép khử để tính toán det theo tốc độ đa thức $\mathcal{O}(n^3)$.

- **Tối ưu không gian:** Thu được lập tức nhờ tích đường chéo chính của khối bậc thang ở trên, nhân với dấu điều hướng $(-1)^s$ từ quá trình Partial Pivoting.

```
# M[i][i] la cac chot cua ma tran bac thang, tich cua chung -> det(U)
det = Fraction((-1) ** s)  # Bat dau voi dau dieu huong
for i in range(n):
    det *= M[i][i]
```

1.3.5 Hàm `rank_and_basis(A)`

Dò các điểm nối nền tảng của không gian vector tạo nên ma trận ban đầu.

- **Cơ sở dòng và cột:** Trích cơ sở dòng từ biến cơ sở của U. Tuy vậy, thuật toán phải tra ngược thông qua index list để trích xuất dữ liệu gốc của cơ sở cột ngay trên trục của ma trận gốc.
- **Cơ sở nghiệm:** Cài đặt tuần tự các điểm neo (biến tự do). Giữ một điểm bằng 1, các điểm khác về 0, giải nghiệm hệ tuyến tính sinh ra các vector trục trực giao chính xác cho toàn bộ không gian nghiệm.

```
# Giai thuat sinh Co so Khong gian nghiem (Null Space Basis, Ax = 0)
for free_j in free_cols:
    special_solution = [Fraction(0)] * n_cols
    special_solution[free_j] = Fraction(1)  # Kich hoat diem Neo thu j

    for i, k in reversed(pivot_indices):
        sum_ax = sum(U[i][j] * special_solution[j] for j in range(k + 1, n_cols))
        special_solution[k] = -sum_ax / U[i][k]

    null_basis.append([str(val) for val in special_solution])
```

1.3.6 Hàm `verify_solution(A, x, b)`

Quá trình kiểm tra được chia làm hai kịch bản rõ ràng và được thẩm định bằng `numpy.allclose(atol=1e-9)`:

- **Trường hợp nghiệm duy nhất:** Chỉ cần ép vòng mảng phân số sang `float` và nhân ngược lại $A \cdot x$. Nếu kết quả xấp xỉ bằng b, chứng tỏ thuật toán tính đúng.
- **Trường hợp vô số nghiệm:** Vì nghiệm trả về dưới dạng `Dictionary` khá phức tạp, nhóm đã phân tách nghiệm tổng quát ra làm hai phần để tự đối chiếu:
  1. *Nghiệm riêng (phần hằng số x_p):* Kiểm tra xem $A \cdot x_p = b$ có khớp không.
  2. *Nghiệm thuần nhất (các vector chứa neo biến tự do v_k):* Đối với mỗi tham số t_k, nhóm tiến hành bóc tách vector hệ số tương ứng và nhân ngược với A. Nếu $A \cdot v_k = \mathbf{0}$, nghĩa là vector đó thực sự khớp với không gian nghiệm.

```python
# Doan code ma nhom dung de kiem tra chat che loi giai vo so nghiem:
# 1. Kiem tra phan hang so lam nhiem vu nghiem rieng: A * x_p = b
x_p = np.array([float(x[i].get('const', Fraction(0))) for i in range(n_cols)])
if not np.allclose(np.dot(A_np, x_p), b_np, atol=1e-9):
    print("-> SAI: Phan hang so khong khop!")
    return

# 2. Kiem tra phan vector tuong ung voi tung bien tu do: A * v_k = 0
for k in free_vars:
    v_k = np.array([float(x[i].get(k, Fraction(0))) for i in range(n_cols)])
    if not np.allclose(np.dot(A_np, v_k), np.zeros(len(b)), atol=1e-9):
        print(f"-> SAI: Vector he so cua bien tu do o t{k+1} khong thuoc Null Space!")
        return
```

1.4 Đánh giá tiêu chí hoàn thành

Dựa trên bảng tiêu chí đánh giá đồ án, nhóm đã rà soát và tự đánh giá mức độ hoàn thành các yêu cầu của Phần 1 như sau:

Tiêu chí	**Mô tả yêu cầu**	**Mức độ hoàn thành**
Cài đặt Gauss cơ bản	Chạy đúng, có kiểm thử rõ ràng	100%
Partial Pivoting	Xử lý đúng khi pivot nhỏ hoặc bằng 0	100%
Tính định thức	Đúng dấu và giá trị	100%
Ma trận nghịch đảo	Gauss-Jordan, kiểm thử $AA^{-1} = I$	100%
Hạng và cơ sở	Đúng rank, trả về cơ sở các không gian	100%
Kiểm chứng	So sánh kết quả với NumPy/SciPy	100%

Bảng 2: Đánh giá tiêu chí hoàn thành - Phần 1

2 Phần 2: Phân Rã Ma Trận và Trực Quan Hóa với Manim

2.1 Chéo hóa ma trận (Matrix Diagonalization)

2.1.1 Cơ sở lý thuyết

Định nghĩa: Một ma trận vuông A cấp $n \times n$ được gọi là chéo hóa được (diagonalizable) nếu tồn tại một ma trận khả nghịch P và một ma trận đường chéo D sao cho:

$$A = P \cdot D \cdot P^{-1} \tag{6}$$

Trong đó:

- $D = \text{diag}(\lambda_1, \lambda_2, \ldots, \lambda_n)$ là ma trận chứa các *giá trị riêng* (eigenvalues) của A trên đường chéo chính.
- $P = [v_1|v_2|\ldots|v_n]$ là ma trận mà các cột là các *vector riêng* (eigenvectors) tương ứng với các giá trị riêng đó.

Điều kiện cần và đủ: Ma trận A cấp n chéo hóa được khi và chỉ khi nó có n vector riêng độc lập tuyến tính. Điều này tương đương với việc tổng số chiều của các không gian riêng (Geometric Multiplicity) phải bằng n. Nếu ma trận có n giá trị riêng phân biệt thì chắc chắn chéo hóa được.

2.1.2 Thuật toán cài đặt

Nhóm triển khai quy trình chéo hóa (trong `diagonalization.py`), ma trận $n \times n$ với $n \leq 5$ thì tự cài đặt thủ công bằng cách **Lặp QR** để rút trị riêng. Nhưng với ma trận $n > 5$ thì nhóm sử dụng hàm `eigvals` của thư viện `numpy.linalg` để rút trị riêng. Cuối cùng sẽ khử Gauss (kế thừa từ phần 1) để xây vector riêng.

A. Xử lý triệt để Ma trận khuyết (Defective Matrix)

Điểm sáng thuật toán của nhóm là hàm `CheckDefective_Eigvals`. Trước khi dựng phương trình vector riêng, thuật toán sẽ đối chiếu Bội hình học (Geometric Multiplicity - GM) và Bội đại số (Algebraic Multiplicity - AM). Nếu $GM < AM$, ma trận lập tức bị từ chối chéo hóa nhằm tránh ma trận P bị suy biến.

```
# Doan code chan dung loi toan hoc the hien tu duy Chat che
AM = rounded_eigs.count(lamda) # Tinh Boi dai so
if AM > 1:
    A_minus_lamdaI = subtract_lambda_I(A, lamda)
```

```
    res_temp = rank_and_basis(A_minus_lamdaI)
    rank = res_temp['rank']
    GM = n - rank  # Tinh Boi hinh hoc tu Hang (Rank-Nullity)

    if GM < AM:    # Phat hien ma tran khuyet, thieu he vector!
        return True
```

B. Phân rã RQ → QR hội tụ trị riêng

Áp dụng phép lặp trực giao Gram-Schmidt biến $A_k = Q \cdot R$, nhân ngược $A_{k+1} = R \cdot Q$. Cấu trúc này dồn phần tử ngoài đường chéo về 0, chừa lại các trị riêng ròng ở đường chéo trung tâm. Các chiều không gian sau đó được quét bằng `rank_and_basis`.

2.2 Phân rã Giá trị suy biến (SVD)

2.2.1 Cơ sở lý thuyết

Phân rã SVD là phương pháp phân tích tổng quát áp dụng cho mọi ma trận $A \in \mathbb{R}^{m \times n}$. Ma trận A được phân tích thành:

$$A = U \cdot \Sigma \cdot V^T \tag{7}$$

Trong đó:

- $U \in \mathbb{R}^{m \times m}$: Ma trận trực giao chứa các vector suy biến trái (các vector riêng của AA^T).
- $V \in \mathbb{R}^{n \times n}$: Ma trận trực giao chứa các vector suy biến phải (các vector riêng của $A^T A$).
- $\Sigma \in \mathbb{R}^{m \times n}$: Ma trận đường chéo chứa các giá trị suy biến $\sigma_1 \geq \sigma_2 \cdots \geq \sigma_p \geq 0$ với $p = \min(m, n)$.

Về mặt hình học, SVD biểu diễn biến đổi tuyến tính qua 3 bước: *Xoay* (V^T) → *Co giãn trục* (Σ) → *Xoay đích* (U).

2.2.2 Phân rã tĩnh bằng phép Xoay Jacobi (Jacobi Eigenvalues)

Thay vì dùng QR, nhóm lựa chọn thuật toán **Jacobi** xử lý riêng biệt cho ma trận đối xứng liên kết $M = A^T A$ vì tính ổn định số học vững chắc, mã nguồn được nhóm xây dựng tại `decomposition.py`.

Hàm hoạt động bằng cách đi tìm các phần tử lệch đường chéo $M[p][q]$, sau đó tính góc xoay θ để đưa nó về 0:

```
# Doan co so luong giac de triet tieu bat ky diem roi lech nao tren hang/cot
if M[p][p] == M[q][q]:
    theta = mth.pi / 4.0
else:
    theta = 0.5 * mth.atan(2.0 * M[p][q] / (M[p][p] - M[q][q]))
cos_t, sin_t = mth.cos(theta), mth.sin(theta)

# Don vi nang tuong lay goc cua M_pq tro ve duong cheo chinh
M_pp = cos_t**2 * M[p][p] - 2*sin_t*cos_t*M[p][q] + sin_t**2 * M[q][q]
M[p][q] = M[q][p] = 0.0 # Clear hoan toan khoi ban do!
```

Sau vòng lặp đưa $M[p][q]$ về 0, M lộ ra ma trận đường chéo gồm các λ_i. Thuật toán kết thúc khi Σ (bằng $\sqrt{\lambda_i}$) và dựng trục suy biến trái U ($u_i = \frac{Av_i}{\sigma_i}$) độc lập hoàn toàn với thư viện.

2.3 Ứng dụng và Trực quan hóa với Manim

2.3.1 Ứng dụng

2.3.2 Trực quan hóa

Để biến các khối công thức đại số tuyến tính trừu tượng trở nên trực quan và dễ tiếp cận, nhóm đã ứng dụng thư viện đồ họa toán học **Manim** để xây dựng công cụ đồ họa dạng video. Mã nguồn kịch bản phim được cấu trúc đối tượng hóa thành các phân cảnh (`Scene`) như sau:

> `Section1_IntroProblem` *Mở đầu bài toán*
>
> Đóng vai trò là phân cảnh mồi (hook/intro), dẫn dắt người xem vào không gian toán học. Khởi tạo một ma trận mẫu đứng trung tâm khung hình. Việc sử dụng các hiệu ứng làm nổi bật từng hệ số giúp định hướng sự tập trung một cách trơn tru.

> `MatrixDiagonalization` *Quá trình Chéo hóa*
>
> Tái hiện từng bước của thuật toán chéo hóa ma trận. Xâu chuỗi tính toán từ việc diễn giải đa thức đặc trưng để tìm *giá trị riêng*, lập hệ phương trình để truy xuất *vector riêng*. Quá

trình gom ma trận chuyển cơ sở P và ma trận đường chéo D được trình bày song song qua các khối sáng màu, chứng minh đồ họa cho công thức $A = PDP^{-1}$.

`Section23_Theory_And_GeometricSVD` *Ý nghĩa hình học của SVD*

Phân cảnh minh họa việc một đường tròn đơn vị và các vector bị biến dạng thành hình ellipse khi nhân trực tiếp với ma trận A. Sau đó video chia tách tiến trình thành 3 nhịp SVD thuần túy: Góc xoay lưới Không gian (V^T), sự co giãn trục định hình ellipse (Σ), và điều hướng chỉnh góc lưới không gian khớp với bản sinh (U).

`SVDAndDiagonalization` *Liên hệ SVD và chéo hóa*

Cầu nối giao thoa bản chất giữa phân rã SVD và chéo hóa. Kịch bản lồng ghép các phương trình biểu diễn lên nhau. Ở trên cùng một phân cảnh, đồ họa hiển thị ma trận tích A^TA vừa được khai triển qua SVD ($V\Sigma^2V^T$) lại vừa bị chéo hóa cổ điển ($P\Lambda P^T$).

`SVDDecompositionVisual` và `LowRankTheory` *Cơ chế xấp xỉ hạng thấp*

Thực hiện lệnh bóc tách đa tầng phương trình cực trị $A = U\Sigma V^T$ thành một chuỗi tổng của các ma trận hạng 1 (A_k). Lớp `LowRankTheory` sau đó triển khai các khối đồ họa logic đóng khung bao bọc để chứng minh đẳng thức tỷ lệ lưu trữ bộ nhớ khi hệ thống chủ động cắt giảm k hạng tử.

`SVDImageCompression` *Ứng dụng nén ảnh SVD*

Ứng dụng giải thuật vào đời sống để phân tách một bức ảnh kỹ thuật số hoàn chỉnh. Khi bậc xấp xỉ k ngày càng tăng lên, các khối ảnh nhiễu được nạp tầng khôi phục dần sắc nét đường bao. Song song đó, thước đo (storage ratio) chạy thời gian thực trên màn hình.

2.4 Đánh giá tiêu chí hoàn thành

Dựa trên các yêu cầu cụ thể đối với phần phân rã ma trận và trực quan hóa, nhóm tự đánh giá kết quả thực hiện Phần 2 như sau:

Tiêu chí	**Mô tả yêu cầu**	**Mức độ hoàn thành**
Lý thuyết phân rã	Trình bày đầy đủ, chính xác công thức	100%
Cài đặt Python	Đúng, có kiểm chứng bằng NumPy	100%
Chéo hóa ma trận	Cài đặt giá trị riêng và chéo hóa	100%
Video Manim	Hoạt ảnh rõ ràng, đủ nội dung	100%

Bảng 3: Đánh giá tiêu chí hoàn thành - Phần 2

3 Phần 3: Giải Hệ Phương Trình và Phân Tích Hiệu Năng

3.1 Các phương pháp cấu hình và giải hệ

Nhóm đã triển khai 3 phương pháp tiếp cận chính để giải hệ phương trình $Ax = b$ trong module `solvers.py`:

1. **Phương pháp trực tiếp - Khử Gauss (Gauss Elimination):** Cài đặt dựa trên thuật toán cốt lõi ở Phần 1 nhưng sử dụng kiểu `float` để mô phỏng chính xác chi phí tính toán trong môi trường thực tế, kết hợp Partial Pivoting.

2. **Phương pháp trực tiếp - Phân rã LU:** Tách ma trận $A = LU$. Giải hệ thông qua hai bước thế tiến ($Ly = b$) và thế lùi ($Ux = y$).

3. **Phương pháp lặp - Gauss-Seidel:** Công thức lặp cho từng phần tử:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i+1}^{n} a_{ij} x_j^{(k)} \right) \tag{8}$$

   Phương pháp lặp này yêu cầu điều kiện hội tụ là ma trận A phải *chéo trội hàng ngặt* (Strictly Diagonally Dominant): $|a_{ii}| > \sum_{j \neq i} |a_{ij}|$.

3.2 Đo lường thời gian thực thi (Performance Benchmark)

Dựa trên script `benchmark.py`, hệ thống tự động sinh các ma trận ngẫu nhiên chéo trội (đảm bảo phương pháp lặp hội tụ) với kích thước $n \in \{50, 100, 200, 500, 1000\}$ để đo thời gian trung bình sau 5 lần chạy độc lập.

Kết quả đo lường cho thấy:

- **Khử Gauss và Phân rã LU:** Thời gian tăng đáng kể khi kích thước ma trận đạt mốc 500 và 1000. Độ phức tạp tính toán thực tế bám sát đường lý thuyết $\mathcal{O}(n^3)$. Trong khi LU nhanh hơn một chút ở pha giải hệ nhiều lần, bước phân rã ban đầu vẫn là cổ chai.

- **Gauss-Seidel:** Thể hiện tốc độ vượt trội ở các ma trận kích thước lớn (nếu thỏa điều kiện hội tụ chéo trội), do chi phí tính toán mỗi vòng lặp chỉ xấp xỉ $\mathcal{O}(n^2)$.

3.3 Đánh giá tính ổn định số học (Condition Number)

Để đánh giá tác động của sai số dấu phẩy động và số điều kiện $\kappa(A)$ lên chất lượng nghiệm, nhóm thực hiện đo đạc sai số tương đối $\frac{||A\hat{x}-b||_2}{||b||_2}$:

- **Ma trận xác định dương (SPD):** Đây là ma trận "Well-conditioned" (số điều kiện nhỏ). Cả 3 phương pháp Gauss, LU và Gauss-Seidel đều cho sai số tương đối rất nhỏ (cỡ 10^{-14} đến 10^{-16}).
- **Ma trận Hilbert ($H_{ij} = \frac{1}{i+j+1}$):** Đây là ma trận nổi tiếng về sự "Ill-conditioned" (số điều kiện tăng theo hàm mũ). Dù chỉ với kích thước nhỏ $n = 10$, số điều kiện đã lên tới $\approx 10^{13}$. Thực nghiệm cho thấy:
  - Phương pháp Gauss và LU: Bị mất mát chữ số có nghĩa (Loss of significance), sai số tăng vọt lên mức không thể chấp nhận được do nhiễu số học.
  - Phương pháp Gauss-Seidel: Không hội tụ do ma trận Hilbert không thỏa mãn tính chất chéo trội hàng.

Kết luận rút ra: Khi giải các bài toán thực tế, không chỉ quan tâm đến tốc độ $\mathcal{O}(n^3)$ mà việc kiểm tra số điều kiện của hệ $\kappa(A)$ là bắt buộc để đảm bảo tính khả tín của nghiệm số.

4 Kết Luận

*Sau một thời gian nghiên cứu và tiến hành cài đặt, Đồ án 1 với chủ đề **"Ma Trận và Cơ Sở của Tính Toán Khoa Học"** đã được nhóm hoàn thiện thành công. Quá trình thực hiện đồ án không chỉ giúp củng cố nền tảng lý thuyết mà còn mang lại những kinh nghiệm thực tiễn quý giá về khoa học thông qua việc lập trình trực tiếp các thuật toán mà không sử dụng các thư viện có sẵn.*

4.1 Tóm tắt kết quả đạt được

Nhóm đã bám sát đề bài và hoàn thành xuất sắc các yêu cầu đề ra, cụ thể:

- **Cài đặt thuật toán cốt lõi:** Triển khai hoàn toàn thủ công các thuật toán: Khử Gauss có chọn phần tử chốt (Partial Pivoting), các thuật toán phân rã như SVD bằng ngôn ngữ Python. Thuật toán đảm bảo độ chính xác nhờ cấu trúc dữ liệu `Fraction`, loại bỏ sai số làm tròn.
- **Giải quyết bài toán thực tế:** Áp dụng thành công phân rã SVD vào kỹ thuật xấp xỉ ma trận hạng thấp (Low-Rank Approximation) để tiến hành nén ảnh kỹ thuật số, hiểu hơn về tầm quan trọng của nén ảnh trong thực tế.
- **Trực quan hóa sinh động:** Xây dựng thành công chuỗi video học thuật bằng công cụ đồ họa **Manim**, biến các công thức đại số trừu tượng thành những đoạn phim trực tiếp giải thích cặn kẽ bản chất hình học của từng bước phân rã SVD và quá trình chéo hóa.
- **Phân tích độ ổn định số:** Đánh giá thực nghiệm sâu sắc về hiện tượng mất ổn định (Ill-conditioned) trên ma trận Hilbert và sự khác biệt về tốc độ hội tụ (Speed Benchmark) giữa các giải thuật.

4.2 Khó khăn gặp phải

Trong hành trình thực hiện đồ án, nhóm đã đối mặt và từng bước giải quyết một số rào cản nhất định:

- **Tính toán khối lượng lớn:** Việc kiên quyết cài đặt bằng Python thuần (không dùng thư viện cấp C như NumPy) đã khiến thời gian thực thi thuật toán $O(n^3)$ trên các ma trận có

kích thước khổng lồ ($n \geq 1000$) tăng cao, đòi hỏi phải tái cấu trúc vòng lặp nhiều lần để giảm chi phí hằng số.

- **Làm chủ công cụ đồ họa Manim:** Việc phải lập trình bằng code kịch bản đòi hỏi nhiều tính toán canh chỉnh tọa độ cho các đối tượng toán học để không bị vỡ bố cục, cùng với đó là việc đồng bộ nội dung, lên kịch bản và tốn tương đối nhiều thời gian để render video thành phẩm.
- **Xử lý ngoại lệ trong tính toán số:** Khó khăn trong việc bắt các lỗi suy biến hệ thống khi xử lý ma trận khuyết (Defective Matrix), cũng như mất mát chữ số phần thực (Loss of Significance) khi thử nghiệm giải hệ với ma trận cực kỳ xấu như ma trận Hilbert.

4.3 Bài học rút ra

Qua khoảng thời gian làm việc, những giá trị cốt lõi mà nhóm đã tích lũy được có thể rút kết thành:

- **Giá trị của sự tối ưu phần cứng:** Hiểu rõ sự chênh lệnh to lớn về mặt hiệu năng khi chuyển đổi các vòng lặp cơ bản.
- **Tư duy toán học và lập trình hướng số liệu:** Nắm vững định nghĩa "Không gian vector" và các phép biến đổi điểm/trục cơ bản, từ đó mới có thể chuyển đổi thành công tư duy phân rã phương trình thành dạng hàm thuật toán có thể giải quyết bài toán lớn hơn của thực tiễn.
- **Kỹ năng làm việc nhóm:** Quá trình làm việc theo các thành phần chức năng (Solver, Demcomposition, Visualization) đã giúp các thành viên củng cố kỷ luật thiết kế phần mềm và rèn luyện kỹ năng giải quyết vấn đề rủi ro chung dưới áp lực của thời hạn yêu cầu.

Tài liệu

[1] Richard Szeliski. *Computer Vision: Algorithms and Applications*, 2nd ed. Springer, 2022.

[2] Gilbert Strang. *Introduction to Linear Algebra*, 6th ed. Wellesley-Cambridge Press, 2023.

[3] Gene H. Golub & Charles F. Van Loan. *Matrix Computations*, 4th ed. Johns Hopkins University Press, 2013.

[4] Manim Community Developers. *Manim – Mathematical Animation Framework*, v0.18. 2024. https://docs.manim.community

[5] 3Blue1Brown. *Essence of Linear Algebra*. YouTube, 2016. https://www.youtube.com/playlist?list=PLZHQObOWTQDPD3MizzM2xVFitgF8hE_ab